

Adapting to the Evolution: Enhancing Intrusion Detection Through Machine Learning in the QUIC Protocol Era

Adam Kadi¹, Lyes Khoukhi², *Senior Member, IEEE*, Jouni Viinikka³, and Pierre-Edouard Fabre

Abstract—The advent of the QUIC protocol may herald a significant shift in the composition of online traffic in the years to come. The transport layer encryption of the QUIC protocol is one of its main evolutions, especially for metadata that was previously transmitted over TCP traffic without encryption. This new protocol has the potential to require significant alterations in future Internet traffic analysis methods and impact network intrusion detection. On the other side, Machine learning has been used in several research projects to identify network intrusions, with positive outcomes. However, we must take into account new evolution of network traffic. In this paper, we propose a new approach that employs supervised machine learning algorithms to identify flows generated by bots interacting with a Web server during a DDoS attack, focusing on the challenges posed by the QUIC protocol and its implications for effective intrusion detection and cybersecurity. Our contribution in this work is divided into three main parts: 1) A guided process with model architecture for emulating and collecting traffic that depict a range of situations our system may encounter; 2) an analysis module that consists on the creation of two labeled datasets, where observations represent the traffic flows detected in PCAP files. We studied the relevance of different features for these datasets, contributing to a thorough understanding of the quality of the data used; 3) a real world experimentation for evaluating the effectiveness of several supervised machine learning algorithms on our datasets. This experimentation allows us to determine which algorithm provides the best prediction results.

Index Terms—Machine learning, DDoS, QUIC, HTTP/3, cybersecurity, intrusion detection system.

I. INTRODUCTION

DDOS attacks are a serious threat to Internet security. These attacks, led by malicious entities, aim to disrupt or disable the online services of a targeted organization. They use networks of compromised devices, often called botnets [1] [2]. One well-known botnet is Mirai, which is primarily composed of Internet of Things (IoT) devices. Once infected, these devices became part of a network controlled by attackers and were used to carry out large-scale DDoS attacks. During a DDoS attack, the attacker uses a botnet to flood the victim's

service with an overwhelming amount of network traffic. This can happen in two ways: overloading the network with a surge of traffic or focusing on the server side, putting stress on its physical resources. This leaves the server unable to effectively respond to legitimate user requests. Over time, cyberattacks have become more sophisticated, as shown in Cloudflare's Q2 2023 DDoS Threat Report.¹ Botnets operating on virtual machines have made these attacks even more powerful than those relying on IoT devices. Cloudflare successfully handled a DDoS attack reaching 71 million requests per second, originating from over 30,000 IP addresses across various cloud providers.

DDoS attacks are closely connected to the protocols governing online communications. Although these communication protocols are essential for digital connectivity, malicious individuals can exploit them to overwhelm target infrastructures, jeopardizing their availability and normal operation. It is important to note that DDoS attacks can vary in sophistication, using a range of techniques and vectors to exploit vulnerabilities in network architecture and application layers. In recent years, DDoS attacks have evolved significantly. In 2010, Akamai² found that 90% of cyber attacks used five main methods: UDP Flood, SYN Flood, ICMP Flood, HTTP GET Flood, and UDP Fragment. Today, these methods only make up 55% of attacks, showing a significant change in how cyber attackers operate. Furthermore, the increasing use of botnets, compromised IoT devices [3], and the availability of DDoS-for-hire services on the dark Web contribute to the growing complexity of the DDoS landscape. Understanding the evolving landscape of DDoS attacks is crucial for developing effective mitigation strategies. As demonstrated by the ever-changing tactics employed by cyber attackers, staying abreast of the latest developments in DDoS attack methods is essential for cybersecurity professionals and organizations. By closely monitoring and analyzing the patterns, techniques, and vectors used in recent DDoS attacks, cybersecurity experts can adapt their defense mechanisms to counter emerging threats effectively. This approach allows for the development of robust and dynamic mitigation solutions that can identify and neutralize new attack methods as they arise.

In this paper, we utilized machine learning techniques to detect and prevent DDoS attacks.

Received 26 March 2024; revised 29 August 2024; accepted 4 February 2025. Date of publication 21 February 2025; date of current version 22 April 2025. The associate editor coordinating the review of this article and approving it for publication was Y. D. Lin. (*Corresponding author: Adam Kadi.*)

Adam Kadi and Lyes Khoukhi are with the ENSICAEN, UNICAEN, CNRS, GREYC, Normandie Université, 14000 Caen, France (e-mail: adam.kadi@6cure.com; lyes.khoukhi@ensicaen.fr).

Jouni Viinikka and Pierre-Edouard Fabre are with 6cure, 14200 Caen, France (e-mail: jvi@6cure.com; pierre-edouard.fabre@6cure.com).

Digital Object Identifier 10.1109/TNSM.2025.3540753

¹<https://blog.cloudflare.com/ddos-threat-report-2023-q2>

²<https://www.akamai.com/blog/security/relentless-evolution-of-ddos-attacks>

Building on our earlier conference paper [4], this study makes several important contributions. The current research significantly extends this analysis. In this study, we shifted our focus to analyzing dynamic bidirectional flows, unlike our previous work, which relied on fixed snapshots of bidirectional traffic captured at specific times. By examining continuous bidirectional flows, we obtain a more detailed and accurate view of network interactions, capturing the real-time exchange of data between clients and servers. This dynamic approach helps us better understand variations in normal traffic and attack behaviors over time, offering more precise insights into traffic patterns. Fixed snapshots, in contrast, might miss transient behaviors and interactions that are important for a complete understanding of network dynamics. We significantly expanded our dataset to about 1 terabyte of data across 132 pcap files, which improves the reliability of our findings. This dataset now includes a wider variety of traffic types, such as video streams, navigation, and download flows for normal traffic, and it also features different attack patterns, including varying request rates per second for GET flood attacks. Our analysis now incorporates a broader range of machine learning algorithms. While our previous work evaluated four algorithms (KNN, SVM, logistic regression, and Random Forest), this study extends the evaluation to eight algorithms, adding MLP, Naive Bayes, Gradient Boosting, and additional logistic regression. We also increased our feature set from 23 to 64 features derived from the detected bidirectional flows. We conducted thorough analyses to assess the importance of each feature in flow classification, using various techniques to measure feature significance. Additionally, we performed an ablation study to determine the optimal number of features needed for satisfactory model accuracy. Finally, our study goes beyond the scope of the conference paper, which focused solely on GET flood attacks. We have included other types of DDoS attacks, such as QUIC Connect and Scan attacks. These attacks involve different methods of targeting the Web server, including overwhelming the connection server. Overall, this work represents a significant advancement from our previous conference paper, offering deeper analysis and greater diversity in data and methods.

Our goal is to develop models capable of distinguishing between malicious botnet activity and regular traffic during an attack. We focused on improving the detection of attacks targeting Web servers using the QUIC protocol, which is employed by the third version of the HTTP protocol [5]. To effectively analyze QUIC traffic within network security frameworks, several advanced approaches and technologies have been proposed:

- 1) *Zeek with Custom Scripts*: Zeek, an open-source network analysis framework, can be augmented with custom scripts to enhance its detection capabilities for QUIC traffic. A notable example is a proof-of-concept script designed to identify both legitimate and malicious QUIC traffic, including traffic associated with command-and-control frameworks like Merlin. This script leverages metadata analysis to distinguish between normal and suspicious traffic patterns, providing a nuanced approach to QUIC traffic analysis.

- 2) *Deep Packet Inspection (DPI) with Encrypted Traffic Intelligence (ETI)*: Ipoque's DPI software integrates Encrypted Traffic Intelligence (ETI) to address the challenges posed by analyzing encrypted QUIC traffic. By employing machine learning and deep learning techniques, this technology enables real-time analysis of traffic patterns, facilitating the identification and classification of encrypted traffic flows. This approach significantly enhances visibility and security by profiling and filtering traffic based on application risks, making it particularly effective in detecting anomalies and potential threats within QUIC traffic.
- 3) *Challenges with Traditional IDS Tools*: Traditional Intrusion Detection and Prevention Systems (IDS/IPS) such as Wireshark, Suricata, and Snort encounter significant challenges in detecting QUIC traffic due to its inherent encryption (TLS 1.3) and reliance on the UDP protocol. These tools often lack the necessary visibility to effectively monitor QUIC traffic, underscoring the need for the development of new detection capabilities or the enhancement of existing systems.

These approaches underscore the necessity for advanced detection techniques and specialized tools to effectively monitor, manage, and secure QUIC traffic within modern network environments.

In our work, we aim to address several specific vulnerabilities and attack vectors associated with QUIC.

Like any protocol, it has its own set of weaknesses that can be exploited in various attack scenarios. One major area we are focusing on is mitigating application-level attacks and Distributed Denial of Service (DDoS) attacks. In a typical DDoS attack, bots from a botnet flood a Web server with a massive number of requests, overwhelming it and causing it to become unresponsive. With QUIC, an attacker might exploit a single connection to send an excessive number of requests. Our approach aims to reduce the effectiveness of these attacks by detecting the malicious behavior of bots flooding a target server with requests. Additionally, we are working to mitigate attacks that involve establishing numerous short-lived connections to the target server. In this scenario, an attacker uses bots to create many connections rapidly, overwhelming the server with a high number of connections in a short period. This can exhaust the server's resources and degrade its performance. By enhancing how QUIC handles connection management and resource allocation, our approach seeks to prevent such connection-based flooding attacks. To combat this, we are incorporating machine learning to detect and differentiate between legitimate and malicious connection attempts. Machine learning models will analyze connection patterns and behavior in real-time to identify and block malicious sources before they can overwhelm the server. Techniques such as connection rate limiting and improved connection state management will be enhanced by these models to better handle high connection churn and prevent resource exhaustion.

Particularly, our main contributions are:

- A guided process with model architecture for emulating and collecting traffic that depict a range of situations our

system may encounter. The traffic captures in pcap format are available on this link [Kaggle Dataset](#).

- An analysis module that consists on the creation of two labeled datasets, where observations represent the traffic flows detected in pcap files. We studied the relevance of different features for these datasets, contributing to a thorough understanding of the quality of the data used. The two datasets and the script with the functions that compute the features are available here [GitHub Repository](#).

- A real world experimentation for evaluating the effectiveness of several supervised machine learning algorithms on our datasets. This experimentation allows us to determine which algorithm provides the best prediction results.

The rest of this paper is organized as follows. Section II presents related works. Section III presents an overview of the three stages of our work. Section IV presents our data acquisition structure. Section V introduces PCAP file processing for flow detection and dataset generation. Section VI is about the results on features and prediction models obtained using Machine Learning algorithms. Section VII concludes the paper and presents future work.

II. RELATED WORK

In this section, we present related research that centers on using machine learning to deal with DDoS attacks. Machine learning-based DDoS mitigation approaches provide various advantages over traditional methods, including the capacity to learn and adapt to new attack situations automatically. When compared to traditional rule-based techniques, they can detect aberrant traffic patterns and identify DDoS attacks more intelligently.

Kumari and Mrunalini [6] proposes a methodology for identifying DDoS attacks using machine learning algorithms. The authors conduct a literature survey to compare existing work in the field and develop a mathematical model to derive the relationship between data packets' inter-arrival time and throughput. The proposed mathematical model aims to estimate the quantitative behavior of the system by deriving the relationship between the inter-arrival time of data packets and the throughput. This model provides a framework for comparing quantitative findings with observational data, particularly focusing on the significant characteristics for determining network attack, such as bandwidth and throughput. They apply logistic regression and naive Bayes algorithms to the CAIDA 2007 [7] dataset to differentiate between normal and attack scenarios. The proposed approach achieved high accuracy in identifying DDoS attacks, with the machine learning model achieving 100% accuracy and the mathematical model achieving 99.75% accuracy.

Golduzian [8] focuses on identifying DDoS attacks, which may cause major harm to network infrastructure. The study utilizes a thorough examination of the CICDDoS2019 dataset [9], which is one of the most comprehensive datasets available for DDoS attack detection. To find the most significant variables for identifying DDoS assaults, the study uses statistical approaches and machine learning algorithms. To choose the attributes for effective identification, data pretreatment

included counting data cleaning, transformation, and reduction. The study underlines the need of picking key variables from a large dataset in order to improve the accuracy and efficiency of DDoS attack detection.

Najafimehr et al. [10] point out that datasets play a pivotal role in DDoS attack detection, serving as the foundation for training and evaluating machine learning algorithms. They explore datasets such as Edge-IIoTset [11], CICDDoS2019 [9], and KDD Cup 99 [12]. While these datasets aim to mimic real-world traffic and attacks, providing accurate representations of network situations, they come with both strengths and limitations. Their vast scale, encompassing millions of records, allows for robust training and evaluation of machine learning algorithms. For instance, the Edge-IIoTset dataset boasts over 20 million records. However, even the most representative datasets have their constraints. The KDD Cup 99, dating from 1999, may not capture the nuances of contemporary network traffic, especially given the evolution of network protocols. Additionally, it might lack diversity in the types of DDoS attacks, failing to cover all possible scenarios. Similarly, the CICDDoS2019, compiled from specific test scenarios, may not be fully representative of all network situations and could lack diversity in the types of DDoS attacks it includes.

In the field of cybersecurity, numerous datasets have been created and utilized for the detection of Distributed Denial of Service (DDoS) attacks in Table I. These datasets typically include a variety of network traffic types, both normal and malicious, and are essential for training and testing machine learning models designed to identify and mitigate such attacks. The current datasets used for DDoS attack detection do not include captures of QUIC traffic or attacks based on this protocol. This is because most of these datasets were developed before or shortly after the introduction of the QUIC protocol. These datasets primarily focus on traditional protocols like TCP, UDP and ICMP, which were the dominant standards at the time of their creation. As a result, they do not account for the specific characteristics of QUIC, such as default encryption and multiplexed streams over UDP, nor do they include scenarios involving attacks specific to this protocol. The absence of QUIC traffic and QUIC-based attacks in these datasets limits their applicability for analyzing and detecting attacks in environments using QUIC.

Sommer and Paxson [23] propose several recommendations to improve future research in anomaly detection using machine learning, including an emphasis on defining the context. This focus is crucial to ensure that anomaly detection systems are not only effective but also suited to operational realities. In our research, we have chosen to concentrate our study on QUIC traffic, specifically for Web applications and DDoS attacks, as this is a topic that is underexplored in the state of the art. Thus, we are focusing on the analysis of bidirectional QUIC flows from a Web server, which will allow us to optimize the accuracy of the results obtained.

In the context of our study, we examined Layer 7 (L7) DDoS attacks, focusing on the challenges associated with their detection. A relevant work in this area is that of Cirillo et al. (2021) [13], which focuses on L7-DDoS attacks that exploit emulation dictionaries. In their research, the authors highlight

TABLE I
A FEW DATASETS FOR DDoS ATTACK DETECTION

Dataset	Year	Format
ISCX2012 [14]	2012	Packet
Booters [15]	2013	Packet
UNSW-NB15 [16]	2015	Packet
NDSec-1 [17]	2016	Packet, logs
CICIDS2017 [18]	2017	Packet
CIC-IDS2018 [19]	2018	Packet
CICDDoS2019 [9]	2019	Packet
Boun DDoS [20]	2019	Packet
LITNET-2020 [21]	2020	Netflow
Edge-IIoTset [11]	2022	Packet
CICIoT2023 [22]	2023	Netflow

how bots use pre-recorded requests to mimic the behavior of legitimate users, making the detection of malicious activities particularly complex. They propose a bot identification algorithm based on an analytical characterization of the interactions between bot clusters, allowing for the distinction between legitimate users and bots, even in scenarios where the emulation dictionaries are partially overlapped. In our own research, we used machine learning algorithms to detect attack flows and normal flows on a Web server. By integrating machine learning techniques, we aim to improve detection accuracy by analyzing the characteristics of requests in real-time. Our approach aligns with the work of Cirillo et al., seeking to identify not only the abnormal behaviors associated with DDoS attacks but also to effectively classify traffic based on its legitimate or malicious nature. Thus, our research contributes to the fight against L7-DDoS attacks by proposing innovative solutions that rely on machine learning methods for proactive and precise detection.

Our study distinguishes itself from previous work by proposing a novel approach to the analysis of Internet traffic, with particular reference to the QUIC and HTTP/3 protocols. Unlike previous research, which has relied on data sets dating back several years and encompassing a variety of attacks, we have developed two data sets based on our own simulated QUIC traffic. Our focus on bidirectional HTTP/3 interactions is a specific feature of our approach, aimed at better understanding the underlying patterns of this emerging protocol. By meticulously extracting several features from connection flows, our aim is to thoroughly evaluate the machine learning models best suited to our context and identify the features most relevant to intrusion detection.

III. FROM RAW DATA TO KNOWLEDGE: A GUIDED PROCESS WITH MODEL ARCHITECTURE

This section presents the proposed a guided process with model architecture. Figure 1 presents an illustrative overview of the three key components of this model architecture, namely Data Acquisition, Flow Extraction, and Data Analysis and Model Generation.

A. Data Acquisition

This step involves emulating interactions between a Web server and different users to capture network traffic data. We employed a virtualization platform called Proxmox. Proxmox VE (Virtual Environment) is an open-source platform for enterprise virtualization. It supports two types of virtualization: KVM and LXC. KVM (Kernel-based Virtual Machine) allows you to run full virtual machines with various operating systems in isolated environments. LXC (Linux Containers) offers lightweight virtualization by sharing the host system's kernel, making it more resource-efficient than KVM. This dual approach provides flexibility for managing both full and lightweight virtualized environments. In Proxmox VE, the structure is organized into three main levels: Datacenter, Node, and Virtual Machine (VM). The Datacenter is the top-level organizational unit, encompassing all the resources and configurations within the Proxmox environment. This includes physical servers (nodes), virtual machines, containers, and storage. It serves as a central management point where you can view and manage all the nodes and VMs in your environment. A Node is a physical server running the Proxmox VE software. Each node can host multiple virtual machines and containers. A Virtual Machine is a software-based emulation of a physical computer. It runs its own operating system and applications, independent of the host node. In Proxmox, VMs are managed under the nodes they run on. You can monitor their status, resource usage, and perform various management tasks like starting, stopping, or migrating them. The node we utilized for our experiments is quite powerful, featuring 16 CPUs and 153.13 GiB of memory. These specifications provide ample processing power and memory capacity, allowing us to run multiple virtual machines and perform complex tasks efficiently during our testing. Here's a detailed overview of the configurations and parameters of this environment.

1) Configuration of Virtual Machines:

• General specifications:

- *CPU Cores*: Each VM is equipped with 1 to 4 vCPUs depending on its specific role.
- *RAM Memory*: The VMs are allocated between 512 MB and 16 GB of RAM.
- *Storage*: Virtual disks range between 20 GB and 200 GB, primarily using the QCOW2 format on SSD datastores for optimal performance.

2) Roles of the VMs:

- *Botnet*: 40 VMs are configured to form a botnet and are used as bots during the attack scenario simulations. Each botnet VM has the following specifications:
 - *CPU*: 1 vCPU
 - *RAM*: 768 MB
 - *Storage*: 32 GB (QCOW2)
- *Target Web Server*: One VM hosts the target HTTP/3 Web server with the following specifications:
 - *CPU*: 4 vCPUs
 - *RAM*: 16 GB
 - *Storage*: 100 GB (SSD)

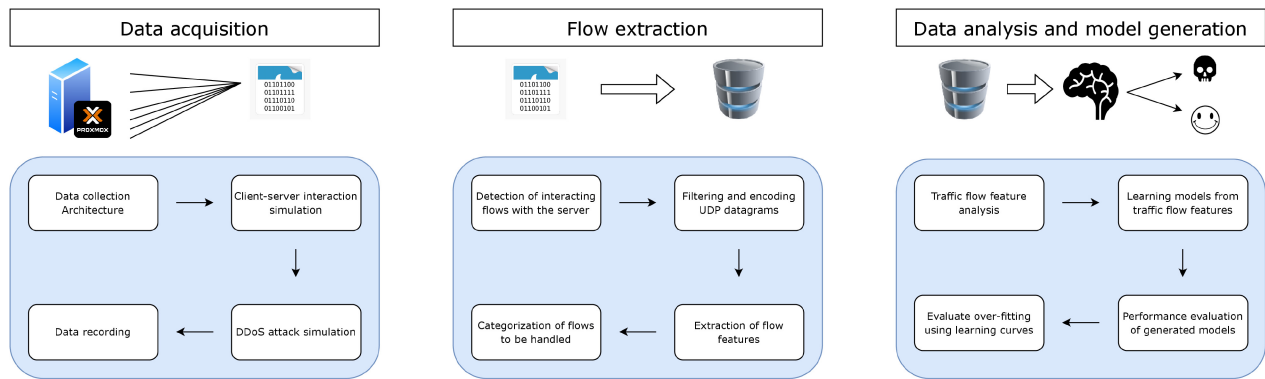


Fig. 1. The three key stages in creating predictive models: collecting data, identifying flows, and applying machine learning algorithms.

- **Background Traffic:** 30 VMs are used to simulate normal background traffic to the target server. Each background traffic VM has the following specifications:
 - CPU: 1 vCPU
 - RAM: 2 GB
 - Storage: 32 GB (QCOW2)
 - **Other VMs:** The remaining 57 VMs are used for various tasks such as management, monitoring, and traffic generation.
- 3) **Network Configuration and Bandwidth:**
- **Network Topology:** The VMs are connected to virtual network bridges (vbr) managed by Open vSwitch (OVS), forming a star topology with a central router.
 - **Network Interfaces:** Each VM uses a Virtio network interface for optimized performance.
 - **Bandwidth Settings:**
 - The network interfaces of the VMs are configured without explicit bandwidth limitations by default, allowing a maximum bandwidth of 1 Gbps per interface. However, limitations can be applied as needed using tools such as `tc` (Traffic Control).

We have configured a virtual machine hosting a Web server accessible via port 443. On this virtual machine, we collect bidirectional Web server traffic. We are using an Nginx Web server, configured with the latest version of the HTTP protocol, known as HTTP/3.³ This version is an upgrade of the HTTP protocol. It was created to improve data transfer performance on the Web by using the QUIC (Quick UDP Internet Connections) transport protocol [24], [25], [26] instead of TCP (Transmission Control Protocol). The Web server hosts files including HTML documents, CSS style sheets, JavaScript code, images, and videos. Utilizing the OpenSSL library, we created an SSL/TLS certificate for the Web server to encrypt traffic communications. To reproduce communication between virtual machines and the Web server, we used Python scripts and the Selenium library.⁴ Selenium is a tool for automating Web browsing, allowing control over actions like typing, clicking buttons, and page navigation. Selenium, using its WebDriver module, enables the emulation of a Web browser like Chrome or Firefox, providing

the capability to automate interactions with Web pages and simulate user behavior. This capability lets developers and testers create scripts that imitate user actions on a website, which is particularly useful for automated testing and data extraction. In our research, we utilized Selenium with Python scripts to generate network traffic between the server and virtual machines acting as clients. When creating our Selenium scripts, we made sure they replicated typical user behavior by incorporating random pauses between each client request, along with actions such as video viewing and form filling. The recorded traffic, considered harmless, will serve as background data for recordings and will be distinguished from potentially harmful attack traffic. For botnet emulation and DDoS attack generation, we leveraged the `quic-go`⁵ library and employed Ansible to efficiently manage and orchestrate the deployment across our network of 40 virtual machines. The `quic-go` library proved useful in creating our attack scripts, utilizing the QUIC protocol as a vector of our DDoS attacks. We carried out our traffic captures in a node on our Proxmox server. The analyzed traffic consists of two distinct network flows defined by specific five-tuple parameters. The first flow corresponds to incoming QUIC traffic towards the Nginx server, while the second represents outgoing traffic from the Nginx server using the QUIC protocol. These parameters, including source and destination IP addresses, source and destination ports, along with the protocol, delineate the unique characteristics of each flow, enabling targeted analysis of network behavior. This node hosted 40 virtual machines. These machines were used to simulate a botnet and launch attacks on the target server. They are devoted to generating interactions between clients and the server, by executing scripts using the Selenium library. These interactions were essential for emulating background traffic categorized as normal.

B. Identification and Analysis of Flows

This module consists of two integrated parts, each playing a distinct but complementary role in the overall detection process. The first part of this module is dedicated to the detection of connection flows. This analysis encompasses two pivotal facets: firstly, the identification of participants, and secondly, the extracting of the network flows generated

³<https://www.nginx.com/blog/our-roadmap-quic-http-3-support-nginx>

⁴<https://selenium-python.readthedocs.io/>

⁵<https://github.com/quic-go>

by these identified participants, providing a comprehensive understanding of their behaviors within the system. The module encodes the data packets, extracting only the essential and relevant information. This encoding method saves storage space and allows greater focus on the crucial details of the packets. Once the packets are encoded, the module records this data, associating it with the user agent that generated them or is the intended recipient. This part was developed using Rust with the libraries `pcap-file`⁶ and `etherparse`⁷ were used in Rust to parse the pcap and encode the packets.

A network flow is an ordered and continuous set of data transmitted between a client and a server within a computer network. This data is encapsulated in UDP (User Datagram Protocol) datagrams, which in turn contain QUIC (Quick UDP Internet Connections) packets. A specific flow is thus made up of an organized set of these UDP datagrams, exchanged between a user and a Web server. Through a series of calculations, the module evaluates various features of each flow. These features include metrics such as frequency of activity, communication patterns, types of data exchanged, and other relevant parameters.

Our method of predicting the actions of the flows produced by virtual machines in our simulations incorporates machine learning. We use supervised methods to classify the interactions of virtual machines on the server based on flows. From the various PCAP files, we produced two labeled datasets using various labeling approaches. We have tagged the generated flows using the IP addresses assigned to each virtual machine. In our simulated environment, we have a direct control over the IP addresses of virtual machines acting as bots launching attacks, as well as those of virtual machines behaving as normal users on the Web server. A comprehensive examination of the retrieved features is one of the steps in our process. We assess each feature's suitability for classifying flows and look for those that significantly contribute to behavior prediction.

C. Data Generation

For our study, we simulated two types of traffic on the Web server: normal client-server interactions and DDoS attacks.

1) *Simulations of Normal Traffic*: We created a wide range of scenarios representing typical Web traffic. This included various requests sent by simulated users and test machines. We adjusted the frequency and intensity of these requests to replicate conditions ranging from moderate activity to periods of high traffic. Some scenarios involved the server handling resource-intensive requests, such as video streaming or accessing multiple Web pages. We also recorded bidirectional traffic to capture a complete view of the data exchanges between clients and the server.

2) *Simulations of DDoS Attacks*: In parallel, we simulated DDoS attacks to assess the server's resilience against threats. Using custom Golang scripts with the `Quic-go` library, we generated malicious traffic aimed at overwhelming the server's

resources—such as CPU, memory, and bandwidth—via the QUIC protocol. The simulated attacks included:

- *HTTP GET Flood*: A bot continuously sends a high volume of HTTP/3 GET requests over a single QUIC connection, overwhelming the server with requests and consuming resources, creating a single intense bidirectional flow between the bot and the server.
- *QUIC Connection Flood*: A bot repeatedly opens numerous QUIC connections to the server, each from different source ports, without sending further data. This strategy is aimed at saturating the server's connection-handling capacity by generating multiple concurrent bidirectional flows, ultimately exhausting its ability to manage new connections.
- *HTTP Scan*: A bot initiates multiple connections to the server, performing a GET request with each connection, and then immediately opens a new connection. This pattern results in several bidirectional flows, each representing an attempt to probe the server's response behavior and potentially uncover vulnerabilities across different source ports.

We carefully generated both normal traffic and DDoS attack scenarios to create a diverse dataset, essential for analyzing our machine learning models.

IV. DATA TRANSFORMATION

In this section, we will discuss our approach to handling UDP datagrams and flows. In Section V-A, we explain how we encode UDP datagrams, where in Section V-B we describe the extraction of flows and lastly in Section V-C, we focus on the features that have been extracted for each flow.

A. UDP Datagram Encoding

In our analysis, each UDP datagram was encoded to retain only the essential information. This encoding process helps us focus on key data and streamline the analysis of QUIC packets. The Figure 2 shows the information encoded on each UDP datagram.

- 1) *QUIC Packet Type*: Each datagram may contain different types of QUIC packets, such as Initial, Handshake, 0-RTT, 1-RTT, Retry, and Version Negotiation. We converted each packet type into a unique binary vector using one-hot encoding. Refer to Table II for the encoding details.
- 2) *Packet Direction*: This indicates whether the datagram is going to or coming from the server.
- 3) *Datagram Size*: The size of the datagram, measured in bytes.
- 4) *Timestamp*: The time when the datagram was captured.
- 5) *Spin Bit Value*: This value is used for specific analyses.

This encoding process transforms the key information into numerical vectors, making it easier to analyze and interpret the data.

B. Extracting Traffic Flows

Flow refers to the continuous, bidirectional movement of data between entities within a network. The idea behind the

⁶<https://crates.io/crates/pcap-file>

⁷<https://crates.io/crates/etherparse/0.13.0/dependencies>

TABLE II
ONE-HOT ENCODING OF DIFFERENT QUIC PACKET TYPES

QUIC Packet Type	Encoding
Initial	[1,0,0,0,0,0]
Handshake	[0,1,0,0,0,0]
0-RTT	[0,0,1,0,0,0]
1-RTT	[0,0,0,1,0,0]
Retry	[0,0,0,0,1,0]
Version Negotiation	[0,0,0,0,0,1]

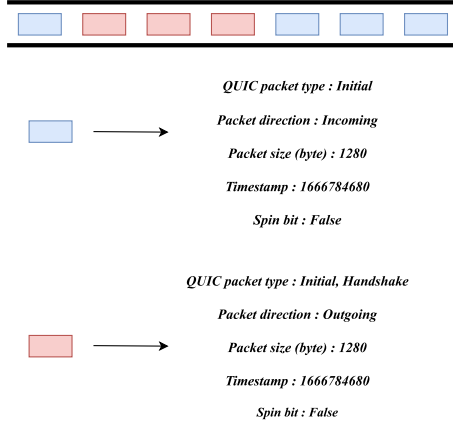


Fig. 2. Information extracted from a UDP datagram during encoding.

concept of flow is the regular, sequential transmission of information from source to a destination. When it comes to differentiating agents within a network, flows play a crucial role. User Agents are software or hardware entities that initiate data flows. This can include Web browsers, applications, or even IoT devices. To identify these agents, we can examine the characteristics of the flows they generate.

However, a complexity emerges when considering NAT (Network Address Translation) gateways. These gateways are often used to enable multiple devices on a local network to access the Internet by sharing a single public IP address. In this scenario, multiple flows from different agents may appear to originate from the same IP address, due to the address translation performed by the NAT gateway as shown in Figure 3. It is difficult to distinguish whether two flows are actually coming from two different agents behind a NAT gateway, or whether they are simply two different flows coming from the same agent. This ambiguity results from the loss of information about the actual IP addresses of the agents downstream of the gateway. We monitor all incoming and outgoing Web server traffic. This method uses an IPv6 address to identify one device and an IPv4 address with a source port number to identify another. By accurately pinpointing the source of DDoS attacks, this packet-level analysis helps implement targeted preventative measures. A connection between a client and a server is viewed as a link between two computer systems, allowing communication and data exchange using appropriate protocols. This ensures quick visibility of users interacting with the server. This method analyzes the server's incoming and outgoing traffic using the QUIC protocol. We detect various bidirectional flows by tracking the source IP address

TABLE III
FEATURES & DEFINITIONS

Features	Definition
Proportion of each type of QUIC packet (6)	Ratio between the number of UDP datagrams containing a specific packet type and the total number of UDP datagrams generated by an flow.
Proportion of the size of each QUIC packet (6)	Ratio of the size (in bytes) of UDP datagrams containing a specific packet type and the total size of all UDP datagrams generated by an flow.
Incoming and outgoing packet size ratio (2)	This feature calculates the ratio of the total size of incoming and outgoing packets from the server to the total size of all packets in the stack. It indicates the proportion of data transmitted from the server compared to the overall data size.
Average throughput (3)	The average throughput is calculated by dividing the total data size by the total communication time. We calculate this metric for the incoming, outgoing and bidirectional flows of the detected agents.
Entropy on QUIC packet type and direction (2)	Entropy is a measure for determining the distribution of packets between incoming and outgoing directions for each list stack generated by an agent, including the entropy of the types of QUIC packets on each packet list.
Packet size variation (3)	The average size variation between consecutive packets for each agent. We calculate this metric for the incoming, outgoing and bidirectional flows of the detected agents.
Packet size standard deviation (1)	The function calculates the standard deviation of packet sizes for each agent.
Statistics related to IAT (mean, standard deviation, skewness, variance) (12)	The IAT (Inter-Arrival Time), or the time between packet arrivals, measures the time between consecutive packet arrivals in a data stream. We calculate statistics related to the IAT (mean, standard deviation, skewness, variance) on the incoming, outgoing, and bidirectional flows of each detected agent.
Ratio of the number of QUIC packets in UDP datagrams (2)	Ratio between the UDP datagrams containing a single QUIC packet and all the UDP datagrams associated with each agent detected and ratio between UDP datagrams containing multiple QUIC packets and all UDP datagrams associated with each detected agent.
The significant time gap in comparison to the total communication time (24)	The count of time gaps detected between each packet using various thresholds relative to the total time of all packets for each identified agent. We calculate this metric for the incoming, outgoing and bidirectional flows of the detected agents.

and source port information of packets that communicate with port 443 of the Virtual Machine's IP address. As it monitors the QUIC packets, it updates the agent list with details from both the source IP address and source port for incoming packets, and the destination IP address and destination port for outgoing packets. This approach allows for comprehensive updates to the agent list, ensuring all bidirectional flows are accurately recorded and monitored.

C. Feature Engineering From Flows

The analysis of flows involves an examination of previously recorded encoded UDP datagrams, constituting the foundation of the feature extraction process. Throughout this procedure, an array of features is calculated to provide a perspective on each data stream. Table III shows the various features that have been calculated for the different flows. One focal point of the feature extraction revolves around the distinctive characteristics associated with the QUIC protocol. In addition, other factors as the amount of data transferred and the size of UDP datagrams added to the range of information collected, thus expanding the set of features studied. Beyond conventional factors like packet frequency and average packet size, specific features pertaining to the behavior and trends of each flow are determined. Furthermore, for each detected flow in the monitored traffic, we aim to predict whether it is malicious or benign to enable the implementation of filtering rules based on IP address and port. The extracted features play a role in machine learning models. By incorporating this rich

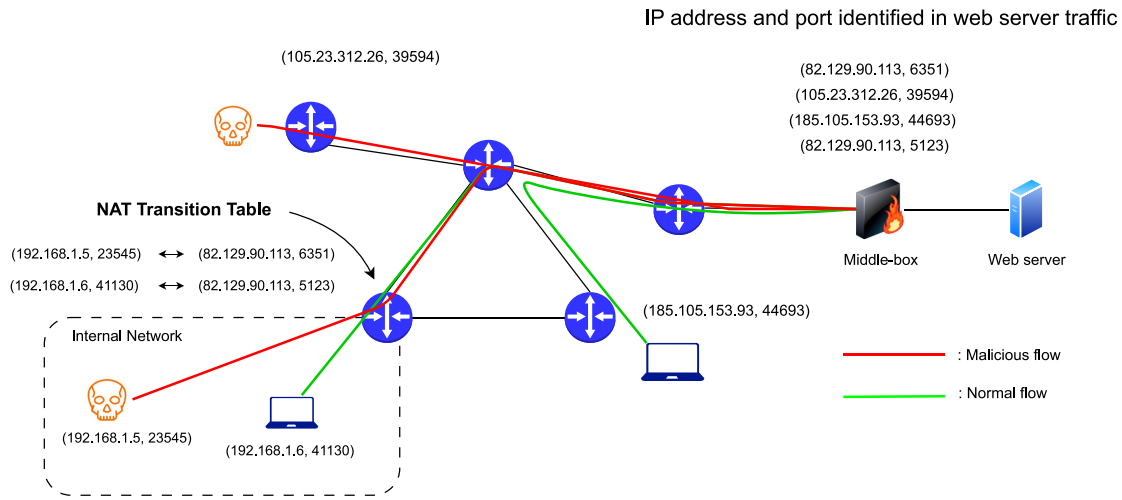


Fig. 3. Representation of a classic DDoS attack, where distributed network traffic converges on a single target.

dataset, the models are empowered to make informed decisions and predictions, thereby contributing to a more intelligent and adaptive network infrastructure. The features calculated relate to various aspects of network traffic, providing a view of the communication between user agents and the Web server.

V. PREDICTIVE INSIGHTS: HARNESSING MACHINE LEARNING ALGORITHMS

After having described our methodology for identifying flows and calculating features in the previous section, we now present our model.

A. Overview of Machine Learning Models Used

In this study, a diverse set of machine learning models was employed to evaluate their performance on the given dataset. The selection of models was aimed at encompassing a broad spectrum of learning paradigms, including linear and non-linear approaches, ensemble methods, and neural networks. This diversity allows for a comprehensive comparison and an understanding of the strengths and weaknesses of each model in the context of the problem at hand. The models considered in this analysis include Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Gradient Boosting Classifier (GBC), Decision Tree (DT), Random Forest (RF), Logistic Regression (LR), Naive Bayes (NB), and a Multi-Layer Perceptron (MLP).⁸

Two distinct datasets in Table IV were generated from the obtained PCAP files. Dataset 1 is labeled to differentiate between two main categories of network traffic flows. The first category includes normal flows, which represent legitimate, bidirectional interactions with the server. The second category includes attack flows, which combine several types of malicious activities, such as GET Flood attacks, QUIC connection attempts, and Scan. These various malicious activities are

⁸The Multi-Layer Perceptron model has four dense layers: 128 neurons *ReLU*, 64 neurons *ReLU*, 32 neurons *ReLU*, and a final output layer with *softmax*. The model is compiled with sparse categorical cross-entropy loss and the adam optimizer.

TABLE IV
INFORMATION ON DATASETS

Information	Dataset	
	Dataset 1	Dataset 2
Number of flow observations	7330	12387
Number of features	52	52
Number of distinct labels	2	4

grouped under a single label to simplify the classification between normal and attack flows.

In Dataset 2, flows were categorized into four distinct classes, each representing a specific type of bidirectional traffic: Normal Flows, Scan, GET Flood, and QUIC Connect. The labels differentiate between flows created by bot-like activities which are GET Flood, HTTP/3 Scan, QUIC Connect, and Normal flows created by Selenium-based navigation. Figure 4 illustrates the two-dimensional visualization of the Dataset 2 using t-distributed Stochastic Neighbor Embedding (t-SNE). This technique effectively captures and represents the high-dimensional relationships between data points in a lower-dimensional space, allowing for a more intuitive understanding of the data structure. Each point corresponds to an individual data instance, color-coded according to its class label: “Normal flow” (red), “QUIC Connect” (blue), “Scan” (yellow), and “GET flood” (green). The separation of the clusters suggests that the t-SNE method successfully distinguishes between different types of network traffic, reflecting the inherent structure and similarities within the dataset. This visualization highlights the method’s ability to reduce dimensionality while preserving the meaningful patterns in the data.

B. Evaluation of Variable Relevance in Datasets

1) *Principal Component Analysis*: This is a widely used dimensionality reduction technique that transforms a large set of correlated variables into a smaller set of uncorrelated variables known as principal components [27]. These principal components are linear combinations of the original variables and are ordered such that the first few components capture

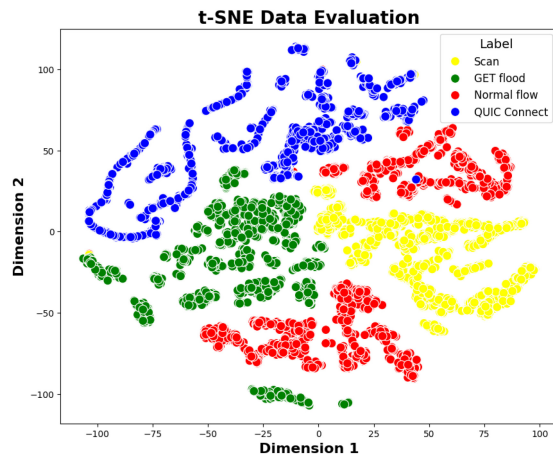


Fig. 4. t-SNE Projection of Traffic Flow Categories.

most of the variability in the data. PCA helps in simplifying the complexity of high-dimensional datasets while retaining most of the relevant information. In the context of this study, Principal Component Analysis (PCA) was applied to the standardized dataset to identify the number of principal components required to explain a significant portion of the variance in the data. The cumulative explained variance by the principal components is depicted in Figure 5. The results indicate that:

- The first principal component accounts for approximately 21.22% of the total variance in the data.
- The cumulative explained variance increases steadily as more principal components are added. For example:
 - The first 4 components together explain about 60.16% of the variance.
 - The first 10 components explain approximately 89.67% of the variance.
 - To explain over 95% of the variance, about 16 components are needed.
 - Nearly all of the variance (99.99%) is captured with around 37 components.

These results suggest that the dimensionality of the dataset can be significantly reduced while still retaining the majority of the information. For instance, by selecting the first 10 to 16 principal components, one can achieve a substantial reduction in dimensionality (from the original number of features) while preserving between 89.67% to 95% of the variance. This reduction simplifies the data without a significant loss of information, making it easier to analyze and visualize.

Analysis in Figure 6 begins with the loading and preparation of a dataset, which is then divided into training and test sets. The data is normalized to ensure that all features are on the same scale. Next, dimensionality reduction is applied via Principal Component Analysis (PCA) to progressively reduce the number of principal components. At each stage of the analysis, the number of principal components is reduced, and for each reduction, several classification models are tested. For each configuration (i.e., for each number of principal components retained), the data are first transformed by PCA to reduce their dimensionality. Each model is then trained

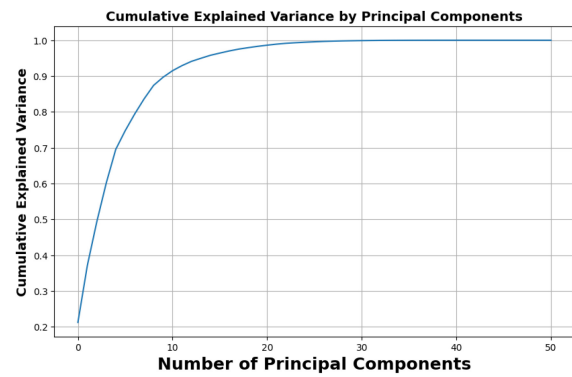


Fig. 5. Cumulative explained variance by principal components.

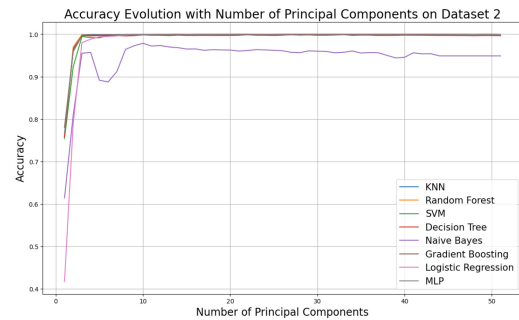
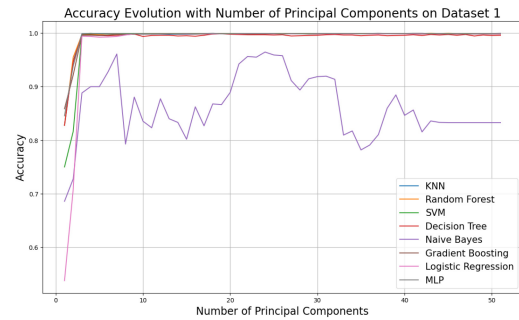


Fig. 6. Accuracy evolution with principal components for two different datasets.

on this new, reduced data set. After training, the model is tested on a separate, reduced test set to assess its accuracy. This measures how well the model classifies data with less information. The aim of this approach is to compare the performance of each model as the complexity of the data decreases (i.e., as the number of principal components used decreases). This allows us to observe which models are most robust to dimensionality reduction, and to determine the optimal point at which reduction begins to adversely affect performance.

Reducing the number of principal components to just 15-20 often preserves high model performance across various algorithms. This approach decreases model complexity and enhances computational efficiency while maintaining robust predictive power, making it particularly advantageous for high-dimensional datasets. We can see that the accuracy of Naive Bayes for both datasets remains slightly lower than that of the other models when the number of principal components is reduced. Naive Bayes is more sensitive to the loss of

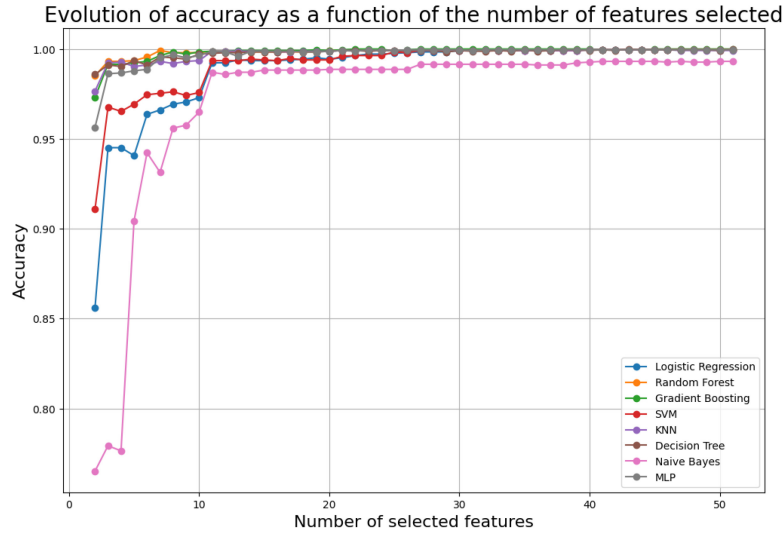


Fig. 7. Impact of ANOVA-Based Feature Selection on the Performance of Classification Models.

information that occurs when the least important components are removed, which may explain the faster drop in accuracy. By reducing the number of principal components, it is possible to reduce model complexity while maintaining high accuracy, as long as the number of principal components remains above a certain threshold (less than 10). PCA is able to condense useful information into a lower-dimensional space, but also highlights the importance of not reducing this number too much to avoid a significant drop in performance.

2) *Variance*: Analysis of variance (ANOVA) is an useful method in data analysis, allowing the examination of significant differences between the means of many groups. In this study, we used ANOVA algorithm on three different datasets, awarding scores to each feature depending on the labels. The goal of this method is to find the features that have a significant influence on the target variable, which are distinguished by the labels assigned. The algorithm calculated F-value scores for each feature, allowing for a quantitative assessment of the significance of differences between label categories. These F-values are indicators for identifying features that significantly contribute to the observed variability in the dataset

$$F\text{-value} = \frac{MSB}{MSW}. \quad (1)$$

Mean Squares Between Groups (MSB), when used in conjunction with ANOVA, quantifies the variability attributable to variations in group means. Its equation is described as:

$$MSB = \frac{\sum_{i=1}^k n_i (\bar{X}_i - \bar{X})^2}{k - 1}, \quad (2)$$

where k is the number of groups, n_i is the number of observations in the group i , $\bar{X}_i$ is the average of the group i , and $\bar{X}$ is the overall average.

Mean Squares Within Groups (MSW) on the other hand, measures the variability within each group. It is defined as:

$$MSW = \frac{\sum_{i=1}^k \sum_{j=1}^{n_i} (X_{ij} - \bar{X}_i)^2}{N - k}, \quad (3)$$

where N is the total number of observations and X_{ij} is the value of observation j in group i .

Figure 7 illustrates the accuracy trends of various machine learning algorithms as the number of selected features increases, with these features ranked by their ANOVA scores in relation to DDoS attack detection. This analysis is part of an ablative study designed to evaluate the impact of progressively removing less relevant features on model performance.

The results reveal a marked improvement in model accuracy with the initial inclusion of the most relevant features. However, beyond a certain point, the accuracy tends to stabilize, suggesting that additional features provide minimal, if any, further enhancement to the models' performance.

Particularly noteworthy are the Random Forest, Gradient Boosting, and MLP models, which achieve high accuracy with a relatively small set of features and sustain this accuracy even as more features are incorporated. This indicates that these models are not only robust but also effective in utilizing the most critical features for DDoS detection, minimizing the risk of overfitting that can arise from adding less relevant features.

In contrast, the Naive Bayes classifier shows a more gradual increase in accuracy, reflecting a greater sensitivity to the number of features used. This behavior suggests that Naive Bayes may require a broader feature set to optimize its performance in the context of DDoS attack detection. Both SVM and Logistic Regression perform well overall but display some variability in accuracy as the number of features increases, potentially indicating a higher sensitivity to the selection of relevant features.

These findings underscore the crucial role of feature selection in enhancing model accuracy for DDoS attack detection. Most of the models achieve their best performance with a limited subset of features, highlighting the importance of rigorous feature selection to mitigate the challenges of high-dimensional data and overfitting—two significant obstacles in applying machine learning techniques effectively. The ablative nature of this study further reinforces the necessity of systematically removing less impactful features to identify the most efficient feature subset for optimal model performance.

TABLE V
FEATURE IMPORTANCE ACROSS DIFFERENT METHODS

Feature	Lasso Importance	Random Forest Importance	GBM Importance
<i>Entropy_packet_direction</i>	0.6765	0.0388	0.0006
<i>Number_UDP_datagramme(outgoing)</i>	-0.0712	0.0768	0.2568
<i>Packet_size_variation(all)</i>	0.2501	0.0095	0.0001
<i>Packet_size_std_deviation</i>	0.2450	0.0407	0.0000
<i>Ratio_spin_bit</i>	-0.2373	0.0884	0.2388
<i>Ratio_no_spin_bit</i>	0.0000	0.1205	0.1249
<i>Iat_incoming(mean)</i>	0.0000	0.0337	0.1904
<i>Number_UDP_datagramme (incoming)</i>	0.0566	0.0395	0.1700

3) *Feature Importance Analysis*: We conducted a comprehensive analysis using three different methods: Lasso regression, Random Forests, and Gradient Boosting Machines (GBM). These techniques provide various perspectives on feature significance and help validate the relevance of the selected attributes in our model.

The results presented in Table V show significant differences in the importance of features according to the evaluation methods used, with, for example, the high importance of *Entropy_packet_direction* in the Lasso model, but negligible importance in the Random Forest and GBM models. Lasso regression assigns coefficients to features, where higher absolute values indicate greater importance. The results reveal that certain features, such as *Entropy_packet_direction*, *Packet_size_variation(all)*, and *Packet_size_std_deviation*, have significant coefficients, confirming their crucial role in classification. In contrast, many other features have coefficients close to zero, suggesting minimal impact on the final model. Random Forests provide importance scores based on the contribution of features to reducing node impurity in trees. Features like *Number_UDP_datagramme(outgoing)*, *Ratio_spin_bit*, and *Iat_incoming(mean)* emerge as the most significant. These findings corroborate the importance of packet statistics and temporal variation features and also highlight features like *Ratio_no_spin_bit* which, while less influential, still hold notable importance. Gradient Boosting Machines (GBM) indicate that *Number_UDP_datagramme(outgoing)*, *Ratio_spin_bit*, and *Iat_incoming(mean)* are among the most influential features. These results reinforce the conclusions drawn from Random Forests, underscoring the robustness of these features across different modeling contexts. The cross-method analysis reveals consistency in the importance of certain features. For instance, *Entropy_packet_direction* and *Ratio_spin_bit* are consistently highlighted as important across different models, reinforcing their relevance. However, features such as *Packet_size_variation(all)* and *Number_UDP_datagramme(outgoing)* show varying levels of importance depending on the method used, emphasizing the need for cross-validation in feature evaluation. Table V presents a summary of the feature importances across the three methods. The table highlights features that are deemed significant by all three methods, such as *Entropy_packet_direction*, *Number_UDP_datagramme(outgoing)*, and *Ratio_spin_bit*.

The variations observed in feature importance across different evaluation methods highlight the sensitivity of models to specific data characteristics. For instance, features like

Entropy_packet_direction show high importance in the Lasso model but are less influential in Random Forest and GBM models. This suggests that the choice of evaluation method can significantly impact the conclusions drawn about feature relevance, and it is crucial to consider multiple approaches to achieve a more robust assessment of important factors.

VI. MODEL EVALUATION

In this section, we detailed our extensive experimentations that we conducted to prove the efficiency and robustness of our proposed approach. We analyzed several models using supervised learning algorithms. Evaluation metrics are crucial for assessing the performance of classification models, especially when errors have varying consequences. In the context of DDoS attack detection and classification, metrics such as false positive rate, detection time, and ROC curve are vital for understanding model performance and its impact on system security. The false positive rate measures how often benign traffic is incorrectly flagged as malicious, which can lead to service disruptions. Minimizing this rate is essential to maintain operational continuity. Detection time, the speed at which traffic is classified as malicious or benign, is critical for promptly mitigating DDoS attacks. The ROC curve and AUC assess the model's ability to distinguish between malicious and benign traffic. A high AUC reflects a model's effectiveness in reducing both false negatives and false positives. Together, these metrics provide a comprehensive evaluation of a model's capability in a security context.

A. Performance Analysis of Supervised Learning Models

We analyzed the false positive rates of several machine learning models for detecting network traffic in Figure 8. The histogram compares the false positive rates across different models for datasets with four labels and two labels. The results show notable differences in false positive rates between models and across datasets. For the dataset with four labels, models such as Random Forest and Gradient Boosting exhibited very low false positive rates, close to zero, indicating high accuracy in avoiding false alarms. In contrast, models like K-Nearest Neighbors (KNN) and Logistic Regression demonstrated higher false positive rates, particularly the latter, which showed a significantly higher rate of 0.025. When the dataset was reduced to two labels, the false positive rates generally increased for most models. KNN and Logistic Regression exhibited the highest false positive rates among the models tested, reflecting a greater frequency of incorrectly

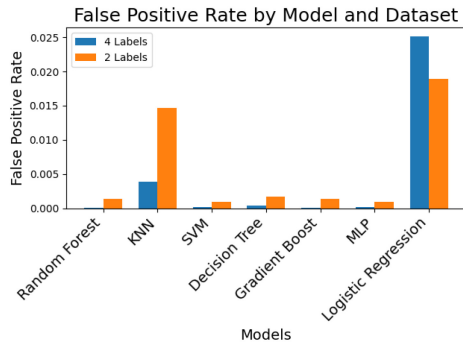


Fig. 8. Comparison of False Positive Rates Across Different Models for 4-Label and 2-Label Datasets.

identifying benign traffic as malicious. On the other hand, models such as SVM and MLP maintained relatively lower false positive rates, similar to their performance on the four-label dataset. Overall, the histogram highlights that while some models are consistently effective at minimizing false positives across different datasets, others vary more significantly. This variation underscores the importance of model selection based on the specific requirements of the application, especially in contexts like network security where false positives can disrupt legitimate services.

In Figure 9, the Receiver Operating Characteristic (ROC) curve is a graphical representation that illustrates the diagnostic ability of a binary classifier system as its discrimination threshold is varied. The curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings, providing insights into the performance of the model across all possible classification thresholds. The Area Under the ROC Curve (AUC) is a summary statistic that quantifies the overall ability of the model to distinguish between the positive and negative classes. An AUC of 1 indicates a perfect model, meaning that the classifier can perfectly separate the positive and negative instances without any error. Conversely, an AUC of 0.5 suggests a model that performs no better than random guessing. In the context of this study, we employed six different machine learning models—K-Nearest Neighbors (KNN), Gradient Boosting, Multi-Layer Perceptron (MLP), Support Vector Machine (SVM), Logistic Regression, Decision Tree, Random Forest, and Naive Bayes—to evaluate the predictive performance on our dataset. Remarkably, the AUC value for each model was found to be 1, indicating that all the models achieved perfect classification on this dataset. This result suggests that the dataset is highly separable, with clear distinctions between the classes. While this outcome is ideal in terms of model performance, it is essential to consider the possibility of overfitting, especially if the models were evaluated on the same data that was used for training.

B. Analyzing Overfitting and Generalization in Supervised Learning Algorithms

One of the major challenges in machine learning is the risk of overlearning, where a model learns to “memorize” training data instead of learning to generalize the essential

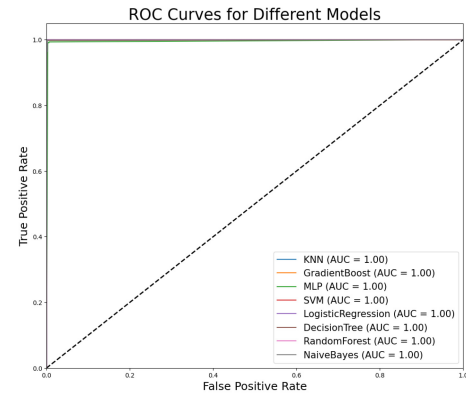


Fig. 9. ROC Curves for Six Machine Learning Models on Dataset 2.

characteristics of normal and malicious flows. This problem is particularly critical in a cybersecurity context, where a model must be able to detect attacks not only on training data, but also on unknown flows that follow similar patterns. Generalization is the ability of a model to make accurate predictions on data it has never seen before. For an intrusion detection system to be effective, it is not enough for the model to correctly identify GET Flood attacks on training data, for example; it must also identify these attacks on future, unknown flows. A model that doesn't generalize well risks missing real attacks or generating numerous false positives, which can have serious consequences for server security.

As explained before, the data used for this analysis comes from a controlled simulation, designed to emulate different types of bidirectional flows with an HTTP/3 Web server. This simulation generated various scenarios to represent different behaviors, but these simulated data may differ from real conditions. So, while the assessment of model overfitting and generalization is valid in this simulated context, the results should be interpreted with caution. They give an indication of the model's performance in this specific configuration, but do not necessarily guarantee equivalent results under real-life conditions. For a complete validation, further tests in real environments would be necessary.

To test if our models are not too specialized to the training data, we used a comprehensive evaluation approach. We first employed repeated cross-validation. This technique involves dividing the data into several groups or “folds.” We train the model on different combinations of these folds and test its performance on the remaining folds. For example, for the SVM model, the F1 scores obtained during cross-validation are very close to those observed on the test sets: 0.9993 for Dataset 1 and 0.9995 for Dataset 2, demonstrating high consistency between cross-validation and final testing.

Next, we measured the models' performance using key metrics such as precision, recall, and the F1 score. Precision indicates how many of the model's predictions are correct, recall shows how many actual positive cases were detected, and the F1 score balances precision and recall.

The results show high performance for all models, with scores very close to 1.0. For example, the Gradient Boosting model achieved a perfect F1 score of 1.0000 on the test sets,

TABLE VI
MODEL PERFORMANCE RESULTS ON BOTH DATASETS

Model	Dataset 1			Dataset 2			Test Set		
	Precision	Recall	F1 Score	Precision	Recall	F1 Score	Precision	Recall	F1 Score
SVM	0.9993 ± 0.0006	0.9993 ± 0.0006	0.9993 ± 0.0006	0.9995 ± 0.0007	0.9995 ± 0.0007	0.9995 ± 0.0007	0.9996	0.9996	0.9996
KNN	0.9991 ± 0.0005	0.9991 ± 0.0005	0.9991 ± 0.0005	0.9995 ± 0.0006	0.9995 ± 0.0006	0.9995 ± 0.0006	0.9992	0.9992	0.9992
Gradient Boosting	0.9992 ± 0.0007	0.9992 ± 0.0007	0.9992 ± 0.0007	0.9994 ± 0.0008	0.9994 ± 0.0008	0.9994 ± 0.0008	1.0000	1.0000	1.0000
Logistic Regression	0.9992 ± 0.0006	0.9992 ± 0.0006	0.9992 ± 0.0006	0.9996 ± 0.0007	0.9996 ± 0.0007	0.9996 ± 0.0007	0.9996	0.9996	0.9996
Decision Tree	0.9986 ± 0.0008	0.9986 ± 0.0008	0.9985 ± 0.0008	0.9994 ± 0.0007	0.9994 ± 0.0008	0.9993 ± 0.0007	0.9996	0.9996	0.9996
Random Forest	0.9994 ± 0.0006	0.9995 ± 0.0005	0.9995 ± 0.0005	0.9996 ± 0.0006	0.9995 ± 0.0007	0.9995 ± 0.0007	0.9996	0.9996	0.9996
MLP	0.9994 ± 0.0006	0.9995 ± 0.0005	0.9995 ± 0.0005	0.9996 ± 0.0006	0.9995 ± 0.0007	0.9995 ± 0.0007	0.9996	0.9996	0.9996
Naive Bayes	0.9910 ± 0.0021	0.9908 ± 0.0021	0.9908 ± 0.0021	0.9966 ± 0.0018	0.9966 ± 0.0019	0.9966 ± 0.0019	0.9933	0.9931	0.9931

which aligns with the cross-validation scores of 0.9992 for Dataset 1 and Dataset 2.

Repeated cross-validation, where data is divided into multiple groups for training and validation, shows that the models are robust and not dependent on any specific data partition. The similar results across different folds, such as the F1 scores of 0.9995 for the Random Forest on Dataset 1 and Dataset 2, confirm that the models generalize well.

The performance on independent test sets, which were not used during training, also supports this generalization capability. For instance, the Logistic Regression model achieved F1 scores of 0.9996 on the test sets, in line with its cross-validation performance. The small differences between the cross-validation and test set performances suggest that the models are well-tuned to capture the underlying data distribution.

These results also indicate that the data used is of high quality, with well-separated classes. However, it is important to note that real-world environments may present additional challenges. Real data may be noisier, classes less well-defined, and attack patterns more varied than those in our simulation.

Therefore, while the results are encouraging, further testing in real-world environments is necessary to ensure that the models maintain their performance under more diverse and unpredictable conditions.

C. Time Complexity Analysis of Machine Learning Methods

In the field of intrusion detection systems (IDS), machine learning methods can play a role in automatically identifying anomalies and malicious activities in computer networks. However, beyond accuracy and detection efficiency, another essential factor requires particular attention: the time complexity of the algorithms used. Time complexity refers to the amount of time needed to execute different stages of a machine learning algorithm, notably the training time and prediction time. Understanding and optimizing this complexity is crucial for several reasons:

- **Real-Time Responsiveness:** In many cases, IDS must react instantaneously to threats to prevent damage. Delays in detection and response can compromise network security.
- **Scalability:** Detection systems often need to process large amounts of data in real-time. Algorithms with high time complexity can become impractical as data size increases.
- **Operational Efficiency:** Computing resources, whether hardware or energy, are limited. A faster and more efficient algorithm allows for optimal use of these resources.

The time complexity of machine learning methods refers to the amount of time required to execute different steps of a machine learning algorithm. This notion is crucial, as it directly impacts the feasibility and effectiveness of solutions based on these algorithms, particularly in contexts where response times are critical, such as in intrusion detection systems (IDS).

Training time is the duration required to train a machine learning model on a given dataset. A high training time can be acceptable in situations where the model needs to be trained only once or infrequently. However, in scenarios where data changes rapidly, methods requiring frequent retraining with long training times can be impractical.

Prediction time is the duration required for a trained model to generate predictions on new data. This includes the processing time needed to prepare input data and execute the model. This time is crucial in real-time applications, where decisions need to be made quickly, such as in network intrusion detection. Short prediction times enable faster responses to potential threats. Total execution time is the sum of the training time and prediction time. It represents the overall time needed to develop, train, and use a machine learning model to make predictions on new data. Understanding the total execution time allows for the assessment of the overall efficiency of a machine learning method and its relevance to a given application.

In Table VII and Table VIII, We have conducted a thorough evaluation of the computational time required for training and prediction across all models used in this work. For this evaluation, we utilized datasets comprising 2477, 4335, and 6193 samples for training and prediction. Additionally, each recorded time represents the average of 100 measurements to ensure accuracy and reliability. The models were implemented and trained using the Python programming language, leveraging the scikit-learn library for their development. The results indicate significant variations in training times across the different models. Simpler models such as Naive Bayes and Decision Trees exhibited the fastest training times, remaining minimal across all sample sizes. This makes them suitable for applications requiring rapid model updates. Conversely, more complex models like Multi-Layer Perceptron (MLP) and Gradient Boosting demonstrated significantly longer training times, which increased with larger sample sizes. These models, although potentially offering higher accuracy, demand more computational resources and time, making them less ideal for scenarios requiring quick turnaround. Intermediate models like Random Forest and Logistic Regression showed moderate training times, striking a balance between complexity and

TABLE VII
TRAINING AND PREDICTION TIME ANALYSIS FOR DIFFERENT MODELS(4-LABEL DATASET)

Model	Training Time (seconds)			Prediction Time (seconds)		
	2477 samples	4335 samples	6193 samples	2477 samples	4335 samples	6193 samples
KNN	0.000425	0.000620	0.001293	0.048238	0.079742	0.097430
MLP	12.941936	14.091911	15.163047	0.125901	0.185777	0.236037
Random Forest	0.385257	0.681592	0.904209	0.014290	0.022311	0.029813
SVM	0.016711	0.031511	0.050547	0.049245	0.087835	0.098428
Decision Tree	0.023715	0.052399	0.074040	0.000178	0.000224	0.000425
Naive Bayes	0.001787	0.002788	0.004505	0.002387	0.003722	0.005936
Gradient Boosting	10.600983	19.807572	25.355199	0.010157	0.016546	0.025733
Logistic Regression	0.543653	0.653236	0.290772	0.000664	0.001023	0.001092

TABLE VIII
TRAINING AND PREDICTION TIME ANALYSIS FOR DIFFERENT MODELS (2-LABEL DATASET)

Model	Training Time (seconds)			Prediction Time (seconds)		
	2477 samples	4335 samples	6193 samples	2477 samples	4335 samples	6193 samples
KNN	0.000380	0.000550	0.001160	0.043400	0.071768	0.087687
MLP	11.650000	12.682000	13.646000	0.113311	0.167199	0.212433
Random Forest	0.346730	0.613433	0.813788	0.012861	0.020080	0.026832
SVM	0.015040	0.028360	0.045492	0.044320	0.079052	0.088585
Decision Tree	0.021340	0.047159	0.066636	0.000160	0.000202	0.000383
Naive Bayes	0.001610	0.002509	0.004054	0.002149	0.003350	0.005342
Gradient Boosting	9.540885	17.826815	22.819679	0.009141	0.014891	0.023160
Logistic Regression	0.489288	0.587912	0.261695	0.000598	0.000921	0.000983

efficiency. Random Forest benefits from parallel processing, which helps manage training times despite its ensemble nature. Support Vector Machine (SVM) showed relatively low training times, especially with the use of a linear kernel, keeping the computational demand in check.

The analysis of the computational complexity of machine learning algorithms helps to understand the efficiency and relevance of different models in specific contexts. Complexity is manifested through several dimensions, including training times, prediction times, and the required computational resources, such as memory and processing capacity.

- *Simple Models (Efficiency and Speed):* Simple models, such as Naive Bayes and decision trees, stand out for their speed in both training and prediction. These algorithms are characterized by relatively low time complexity, mainly due to the nature of their operations. For example, Naive Bayes relies on basic probabilistic calculations and assumptions of conditional independence, which reduces the number of required computations. Similarly, decision trees operate by successive splits of data based on optimal criteria, an operation that remains fast even for larger datasets.

These models are particularly advantageous in scenarios where response time is crucial, such as real-time applications. For instance, in network intrusion detection, a model's ability to instantly react to anomalies is essential. The low computational complexity of these models makes them ideal for environments with limited resources, where speed and efficiency are priorities.

- *Intermediate Models (A Balance Between Complexity and Performance):* Intermediate models, such as Random Forest and logistic regression, offer a trade-off between simplicity and complexity. Random Forest, for example, combines several decision trees to improve the robustness and accuracy of the model, but this increases computational complexity due to the need to manage multiple models in parallel. However, using techniques like parallel processing helps manage this complexity, offering reasonably efficient performance for tasks where moderate accuracy is sufficient.

Logistic regression, on the other hand, uses a sigmoid function to model the probability of class membership. Although training may require successive iterations to converge, the complexity remains manageable due to the model's linear nature. These algorithms are suitable for situations where a good balance between accuracy and time efficiency is needed, such as in recommendation systems or risk scoring, where computations must be quick but reliable.

- *Complex Models (Accuracy at the Expense of Complexity):* More complex models, such as the Multi-Layer Perceptron (MLP) and Gradient Boosting, are designed to maximize predictive accuracy, often at the expense of speed and computational efficiency. The MLP, which belongs to the family of neural networks, involves iterative calculations and the adjustment of multiple parameters through hidden layers. This process, while powerful for capturing nonlinear relationships in data, is

costly in terms of time and resources, as evidenced by the longer training times observed.

Gradient Boosting, meanwhile, combines several weak models (typically decision trees) to create a strong model. Each successive model is trained to correct the errors of the previous one, which gradually increases complexity. This approach, while highly effective for demanding predictive tasks, requires significantly longer training times and more computational resources.

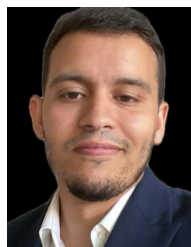
These models are better suited for offline analysis, where priority is given to accuracy rather than speed. They are often used in contexts where decisions must be made with a high degree of certainty, such as in fraud detection or financial market forecasting, where errors can be costly.

VII. CONCLUSION

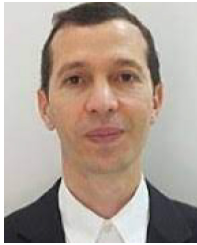
In this study, we examine how DDoS attacks are evolving with the increasing use of the QUIC protocol. While QUIC offers advantages such as faster, more secure connections, it also presents new challenges for the detection of malicious traffic due to its encrypted nature. To address these challenges, we have created a structured approach that involves collecting traffic data and using it to train supervised learning models. We tested several algorithms, including logistic regression, KNN, Random Forest and SVM, to see how well they could detect DDoS attacks targeting QUIC. Our results showed that some machine learning methods are effective in identifying bidirectional flows of attacks. While our research provides valuable insights into the detection of DDoS attacks in QUIC environments, it also highlights the need for further research. As network protocols and attack methods evolve, our detection strategies must adapt to ensure network security in the face of new threats. Detection systems must be continually refined to stay ahead of evolving threats.

REFERENCES

- [1] M. Bailey, E. Cooke, F. Jahanian, Y. Xu, and M. Karir, "A survey of botnet technology and defenses," in *Proc. Cybersecurity Appl. Technol. Conf. Homeland Security*, 2009, pp. 299–304, doi: [10.1109/CATCH.2009.40](https://doi.org/10.1109/CATCH.2009.40).
- [2] S. N. Thanh Vu, M. Stege, P. I. El-Habr, J. Bang, and N. Dragoni, "A survey on botnets: Incentives, evolution, detection and current trends," *Future Internet*, vol. 13, no. 8, p. 198, 2021. [Online]. Available: <https://doi.org/10.3390/fi13080198>
- [3] M. Wazzan, D. Algazzawi, O. Bamasaq, A. Albeshri, and L. Cheng, "Internet of Things botnet detection approaches: Analysis and recommendations for future research," *Appl. Sci.*, vol. 11, no. 12, p. 5713, 2021. [Online]. Available: <https://doi.org/10.3390/app11125713>
- [4] A. Kadi, L. Khoukhi, J. Viinikka, and P.-E. Fabre, "Machine learning for QUIC traffic flood detection," in *Proc. Global Inf. Infrastruct. Netw. Symp. (GIIS)*, 2024, pp. 1–6, doi: [10.1109/GIIS59465.2024.10449925](https://doi.org/10.1109/GIIS59465.2024.10449925).
- [5] M. Bishop, "HTTP/3," Internet Eng. Task Force, RFC 9114, Jun. 2022. [Online]. Available: <https://www.rfc-editor.org/info/rfc9114>
- [6] K. Kumari and M. Mrunalini, "Detecting denial of service attacks using machine learning algorithms," *J. Big Data*, vol. 9, no. 1, p. 56, 2022. [Online]. Available: <https://doi.org/10.1186/s40537-022-00616-0>
- [7] V. Shirsath, May 2023, "CAIDA UCSD DDoS 2007 attack dataset," Dataset, IEEE Dataport. [Online]. Available: <https://dx.doi.org/10.21227/dvp9-s124>
- [8] A. Golduzian, "Predict and prevent DDOS attacks using machine learning and statistical algorithms," 2023, *arXiv:2308.15674*.
- [9] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Security Technol. (ICCSST)*, 2019, pp. 1–8, doi: [10.1109/CCST.2019.8888419](https://doi.org/10.1109/CCST.2019.8888419).
- [10] M. Najafimehr, S. Zarifzadeh, and S. Mostafavi, "DDoS attacks and machine-learning-based detection methods: A survey and taxonomy," *Eng. Rep.*, vol. 5, no. 12, 2023, Art. no. e12697, doi: [10.1002/eng2.12697](https://doi.org/10.1002/eng2.12697).
- [11] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, Jan. 2022, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications: Centralized and federated learning," Dataset, IEEE Dataport. [Online]. Available: <https://dx.doi.org/10.21227/mbc1-1h68>
- [12] S. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. Chan, 1999, "KDD Cup 1999 Data," Dataset, UCI Machine Learning Repository. [Online]. Available: <https://doi.org/10.24432/C51C7N>
- [13] M. Cirillo, M. D. Mauro, V. Matta, and M. Tambasco, "Application-layer DDOS attacks with multiple emulation dictionaries," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2021, pp. 2610–2614, doi: [10.1109/ICASSP39728.2021.9413570](https://doi.org/10.1109/ICASSP39728.2021.9413570).
- [14] A. Shiravi, H. Shiravi, M. Tavallaei, and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark datasets for intrusion detection," *Comput. Secur.*, vol. 31, no. 3, pp. 357–374, 2012.
- [15] R. Singh, H. Kumar, and R. Singla, "A reference dataset for network traffic activity based intrusion detection system," *Int. J. Comput. Commun. Control*, vol. 10, no. 3, pp. 390–402, 2015.
- [16] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, 2015, pp. 1–6, doi: [10.1109/MilCIS.2015.7348942](https://doi.org/10.1109/MilCIS.2015.7348942).
- [17] F. Beer, T. Hofer, D. Karimi, and U. Bühler, "A new attack composition for network security," in *Proc. 10th DFN-Forum Kommunikationstechnologien*, 2017, pp. 30–31.
- [18] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. Int. Conf. Inf. Syst. Security Privacy*, 2018, pp. 108–116.
- [19] "A realistic cyber defense dataset (CSE-CIC-IDS2018)," 2018. [Online]. Available: <https://registry.opendata.aws/cse-cic-ids2018>
- [20] D. Erhan, Oct. 2019, "Bogaziçi university DDoS dataset," Dataset, IEEE Dataport. [Online]. Available: <https://dx.doi.org/10.21227/45m9-9p82>
- [21] R. Damasevicius et al., "LITNET-2020: An annotated real-world network flow dataset for network intrusion detection," *Electronics*, vol. 9, no. 5, p. 800, 2020.
- [22] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023. [Online]. Available: <https://doi.org/10.3390/s23135941>
- [23] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security Privacy*, Oakland, CA, USA, 2010, pp. 305–316, doi: [10.1109/SP.2010.25](https://doi.org/10.1109/SP.2010.25).
- [24] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," Internet Eng. Task Force, RFC 9000, May 2021. [Online]. Available: <https://www.rfc-editor.org/info/rfc9000>
- [25] M. Thomson and S. Turner, "Using TLS to secure QUIC," Internet Eng. Task Force, RFC 9001, May 2021. [Online]. Available: <https://www.rfc-editor.org/info/rfc9001>
- [26] J. Iyengar and I. Swett, "QUIC loss detection and congestion control," Internet Eng. Task Force, RFC 9002, May 2021. [Online]. Available: <https://www.rfc-editor.org/info/rfc9002>
- [27] M. Greenacre, P. J. F. Groenen, T. Hastie, A. I. D'Enza, A. Markos, and E. Tuzhilina, "Principal component analysis," *Nat. Rev. Methods Primers*, vol. 2, p. 100, Dec. 2022, doi: [10.1038/s43586-022-00184-w](https://doi.org/10.1038/s43586-022-00184-w).



Adam Kadi received the bachelor's degree in computer science and the master's degree in IT systems security from the University of Caen Normandie, France. He is currently pursuing the Ph.D. degree with the CIFRE (Industrial Agreement for Training through Research), ENSICAEN, Normandie Université, the GREYC laboratory (CNRS), and the company 6cure, specialized in protection against DDoS and DNS attacks. His thesis focuses on artificial intelligence for DDoS attack mitigation, aiming to develop advanced solutions to address growing cybersecurity challenges.



cloud networking, 5G/SDN, IoT/V2X, and CPS.

Lyes Khoukhi (Senior Member, IEEE) received the Ph.D. degree in electrical and computer engineering from the University of Sherbrooke, Canada, in 2006. From 2007 to 2008, he was a Postdoctoral Researcher with the Department of Computer Science and Operations Research, University of Montreal. He is currently a Full Professor with the ENSICAEN, Normandie University, GREYC CNRS. His current research topics are in the field of cybersecurity, attack detection, and performance evaluation in advanced networks like



Jouni Viinikka received the Ph.D. degree in computer science from the University of Caen, France. He is currently the Head of Research with 6cure. Before co-founding 6cure, he worked at Orange Labs as a research engineer. He has contributed to several national and international research projects. Initially his work focused on intrusion detection and alert correlation. His current research interests are in distributed denial of service - in attacks, attackers and defenses - and he is looking the problem from a practical angle, bridging academic research and real world solutions.



been actively involved in French and European research projects, addressing key challenges in cybersecurity. His work spans decision support systems for selecting mitigation strategies, collaborative detection mechanisms against DDoS attacks, and advanced techniques for identifying attacks on encrypted data. More recently, he has been exploring the use of Artificial Intelligence for DDoS attack detection and mitigation. Through these initiatives, he continues to contribute to the development of innovative security solutions that strengthen network resilience against evolving cyber threats.

Pierre-Edouard Fabre received the Ph.D. degree focusing on the mitigation of volumetric DDoS attacks using network resources in 2018. He is a Research Engineer specializing in cybersecurity, with 6cure, a French company dedicated to protecting networks against Distributed Denial of Service (DDoS) attacks and enhancing overall network security. His research explored novel approaches to traffic analysis and damage control, aiming to differentiate malicious traffic from legitimate users while preserving service availability. Since then, he has